Technical reference manual

1.0.0

EtherBench

Programmer

To emulate the SWD or JTAG signals, we are using a pair of SPI peripherals whose pins can be multiplexed to USART buses. This hardware topology allows for the complete emulation of the ARM Specification.

Peripherals

The peripheral configuration is mapped as follows:

Mode	`SPI2`	`SPI3`	`USARTx`
SWD		**X**	Optional*
JTAG	**X**	**X**	

> **USART Usage**
>
> As permitted by the SWD specification, an additional pin (**SWO**) can be used for high-speed trace logging. This asynchronous pin can be directly routed to a raw USART peripheral, justifying its potential allocation in this mode.

The `SPI3` peripheral is strictly configured as a Master to generate the reference clock (TCK). The `SPI2` peripheral operates exclusively in JTAG mode as a Slave. On the PCB layout, the `SPI3` clock pin is physically tied to the `SPI2` clock pin. This hardware constraint must be strictly respected during firmware development.

The `SPI3` is configured as half duplex, due to the direction changes. The `SPI2` remains as full duplex to handle the JTAG.

Pin	Source Peripheral
`TCK` / `SWCLK`	`SPI3_SCLK`
`TMS` / `SWDIO`	`SPI3_IO`
`TDI`	`SPI2_MOSI`
`TDO`	`SPI2_MISO`
`RST`	`GPIO`
`PRESENT`	`GPIO`

Consequently, for any transfer to occur on `SPI3`, an active transfer **MUST** be simultaneously running on `SPI2`. Furthermore, data must be preloaded into the `SPI3` buffers before initiating the master clock.

Exploitation

The SWD protocol can be particularly tedious to implement since its transfer widths are not byte-aligned. Leveraging the latest generation of ST peripherals significantly mitigates this issue, as the hardware transfer size can be dynamically adjusted between 4 and 32 bits. We chose a default transfer size of 16 bits. This ensures reliable data shifting while providing sufficient margin to adapt the transfer size without violating the protocol timings. A dedicated algorithm computes the required number of 16-bit transfers and dynamically adjusts the frame size for the final residual bits.

Conversely, the JTAG protocol is more straightforward to implement. It relies on continuous, larger frames that can be processed as large byte arrays shifted directly by the DMA.

In all cases, due to the critical timings and turnaround quirks inherent to these protocols, the programmer engine must run as a dedicated, high-priority task. It may rely on polling for ultra-short transfers. As a result, lower-priority tasks might experience slightly increased latencies while a programming sequence is actively running.

RTOS Behavior

As highlighted above, the programmer holds a critical position within the global architecture. To guarantee strict signal timings, the programmer task is authorized to dynamically disable RTOS preemption for specific, time-critical operations. However, this lock is never maintained during long, DMA-backed transfers.

Because programming latencies are generally tighter than standard bus operations, the payload data does not flow through the main Router. Instead, the CMSIS-DAP front-end server communicates directly with the programmer queues.

Other Operation Modes

From a broader perspective, the programmer block is optimized for large data transfers, utilizing its own dedicated memory pool and queues. This architecture allows it to handle more generic operations natively—such as direct EEPROM read/write sequences—without requiring the execution of a custom script. Should a specific I/O profile be unsupported natively, it will fallback to the standard `Hardware IO` task, trading off latency for flexibility.

Accessing Other Peripherals

As depicted on the master architectural schematic, the programmer *can* interface with the main Router and the `Hardware IO` task. This cross-linking ensures future scalability, enabling the support of protocols that cannot be handled by the SPI pair alone. A prime example is the ICSP protocol (AVR), which may require custom bit-banging sequences or specific target voltages.